

WAF Rule Engine

Managed and Customer-Defined Policies, Explained in Simple Terms

[Summary](#) · [Architecture Diagram](#) · [Components](#) · [Workflows](#) · [Results](#)

The project statement being explained

“Shipped a commercially operated WAF rule engine for managed and customer-defined policies (ModSecurity/OWASP CRS-style URI, query-string, header, cookie, and body inspection, normalization, regex and signature matching, anomaly scoring, exclusions, and allow, log, challenge, rate-limit, or block actions), cutting unnecessary rule evaluations 40% through staged evaluation, compiled and cached match structures, request-size safeguards, and per-rule telemetry.”

Note: the design described here is a typical, industry-standard way to build this kind of rule engine. Rule counts, per-stage numbers and the breakdown of the 40% reduction are illustrative examples consistent with the headline result, not measurements from a particular company's system.

Contents

(If page numbers show as blank, right-click the table above in Word and choose “Update Field”.)

1. Overall Summary in Simple Terms

The one-paragraph version

A **WAF (Web Application Firewall)** protects websites by reading every incoming web request and checking it against a list of **rules** that describe attacks – for example “a search box should never contain SQL commands”. I built and shipped the **rule engine**: the part that reads the request (its web address, query, headers, cookies and body), cleans it up so attackers cannot disguise anything, checks it against thousands of rules, adds up a “danger score”, and then chooses one action: **allow, log, challenge, rate-limit or block**. Rules come from two places: **managed rules** written by our security team (in the style of the well-known ModSecurity / OWASP Core Rule Set) and **custom rules** written by each customer. It is a commercial product, so it runs for many paying customers at once. By checking cheap things first, skipping rules that could not possibly match, re-using pre-built matching structures, capping how much of a huge request is inspected, and measuring every rule, I cut **unnecessary rule evaluations by 40%**.

1.1 A simple analogy: airport security

- **Rules = the list of forbidden items.** Some lists come from the airport authority (managed rules); some are added by a particular airline (customer rules).
- **Target extraction = opening every pocket and bag** – the ticket (URL), the passport (headers), small pockets (cookies) and the suitcase (body).
- **Normalization = unwrapping disguises** – taking items out of gift wrap so a knife cannot pass as a present.
- **Staged evaluation = fast lanes.** Crew with badges walk straight through (allow list). A traveller with only a small handbag does not need the oversized-luggage check (scope filter). A quick X-ray first, and only suspicious bags are opened by hand (prefilter, then full match).
- **Anomaly scoring = adding up suspicion.** One odd item may be fine; several odd items together trigger action.
- **Actions = allow, note it down (log), ask more questions (challenge), slow the queue (rate-limit), or stop them (block).**
- **Exclusions = known exceptions**, such as “chefs may carry knives through the staff entrance”.
- **Request-size safeguards = a maximum bag size**, so one enormous trunk cannot hold up the whole queue.
- **Per-rule telemetry = keeping records for every check**, so the airport knows which checks catch real threats and which just waste time.

1.2 What each phrase of the statement means

Phrase	Plain-English meaning
Shipped a commercially operated WAF rule engine	Built and released the rule engine as part of a paid product that runs live, 24/7, for many customers.

Phrase	Plain-English meaning
Managed and customer-defined policies	Two kinds of rules: ones maintained by the vendor's security team, and ones each customer writes for their own site.
ModSecurity / OWASP CRS-style	Follows the model of ModSecurity (the classic open-source WAF engine) and the OWASP Core Rule Set (the most widely used open rule collection).
URI, query-string, header, cookie and body inspection	Every part of a web request is examined: the address, the parameters after "?", the headers, cookies, and the content sent in the body (forms, JSON, XML, file uploads).
Normalization	Decoding and cleaning request data (for example turning %27 back into ') so disguised attacks become visible.
Regex and signature matching	Checking data against text patterns (regular expressions) and known attack fingerprints (signatures).
Anomaly scoring	Each matching rule adds points; the request is acted on only when the total passes a threshold. This reduces false alarms.
Exclusions	Customer- or vendor-defined exceptions that switch off specific rules for specific parts of a site where they cause false alarms.
Allow, log, challenge, rate-limit or block actions	The five possible outcomes for a request.
Cutting unnecessary rule evaluations 40%	40% fewer rule checks that could never have matched (or were not needed) – less CPU work per request, with the same protection.
Staged evaluation	Running checks in order from cheapest to most expensive, and stopping or skipping as early as possible.
Compiled and cached match structures	Turning rules into fast, ready-to-use data structures once, then re-using them for every request instead of rebuilding them.
Request-size safeguards	Limits on how big each part of a request can be and how much of it is inspected, so huge requests cannot overload the engine.
Per-rule telemetry	Measuring each rule individually: how often it runs, how often it matches, how long it takes, and how often it causes false positives.

1.3 The result at a glance

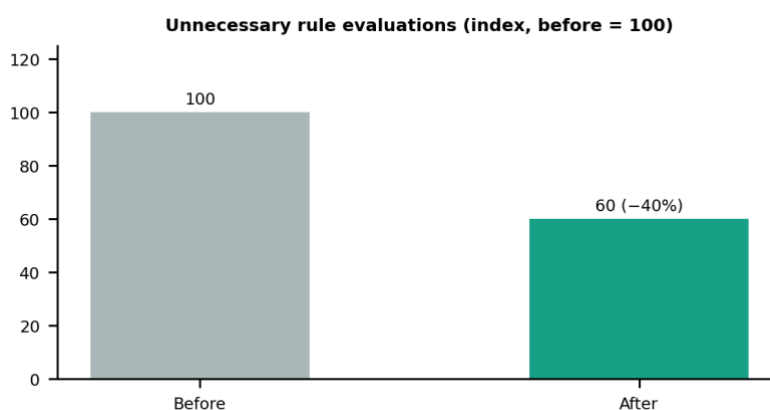


Figure 5 – Unnecessary rule evaluations before and after (relative index)

A 40% cut in unnecessary evaluations means less CPU per request, lower and more predictable inspection latency, room to add more protective rules without slowing sites down, and lower infrastructure cost – **with no loss of detection**, because the only evaluations removed are ones that could not have changed the outcome.

2. Background Concepts in Plain Language

2.1 Key terms

Term	Simple explanation
WAF	Web Application Firewall – software that inspects web requests and blocks attacks against websites and APIs.
Rule	One instruction: “look at these parts of the request, clean them this way, check for this pattern, and if found do this”.
Policy	A customer’s complete security setup: which rule sets are on, custom rules, exclusions, thresholds and actions.
ModSecurity	A widely used open-source WAF engine that defined the classic rule language (SecRule).
OWASP CRS	OWASP Core Rule Set – a free, community-maintained set of rules covering common attacks such as SQL injection and XSS.
Target / variable	Which part of the request a rule looks at, e.g. ARGS (all parameters), REQUEST_HEADERS:User-Agent, REQUEST_COOKIES, REQUEST_BODY.
Transformation	A clean-up step applied before matching, e.g. urlDecode, lowercase, htmlEntityDecode, removeNulls, compressWhitespace.
Operator	How the rule checks the value: @rx (regex), @pm (list of phrases), @contains, @streq, @detectSQLi, @detectXSS, @ipMatch, numeric comparisons.
Phase	When in the request’s life a rule runs: 1 = request headers, 2 = request body, 3/4 = response headers/body, 5 = logging.
Signature	A known fingerprint of a specific attack or exploit (e.g. a CVE exploit string).
Anomaly score	The running total of points a request collects from matching rules.
Paranoia level	How strict the rule set is (1 = only very reliable rules, 4 = very strict). Higher catches more but raises false positives.
False positive (FP)	A legitimate request wrongly flagged as an attack.
Exclusion	An exception that stops a specific rule from checking a specific field or URL.
Regex / ReDoS	Regex = a text-pattern language. ReDoS = a crafted input that makes a badly-written regex take an extremely long time.
Rule evaluation	One rule checked against one target value. A request can trigger thousands of these.
Unnecessary evaluation	An evaluation that could not possibly match or change the result – e.g. a body rule on a request with no body, a rule excluded for that path, or a regex whose required words are not present.

2.2 Anatomy of a rule

Every rule – managed or customer-defined – is made of the same parts. Here is a typical example in ModSecurity/CRS-style form and what each part means:

Example rule (simplified)

```
SecRule ARGS|REQUEST_COOKIES "@rx (?i)union[\s{}+select" "id:942100, phase:2,
t:urlDecodeUni, t:lowercase, severity:CRITICAL, tag:attack-sqli, setvar:tx.sql_score+=5, msg:'SQL
injection UNION SELECT'"
```

Part	Example value	Meaning in simple terms
Targets	ARGS REQUEST_COOKIES	Look in all parameters and all cookies.
Operator	@rx (?i)union[\s{}+select	Look for the words “union” then “select” (any case, separated by spaces or a bracket).
ID	942100	A unique number, used for exclusions, logs and telemetry.
Phase	2	Run after the body has been received.
Transformations	urlDecodeUni, lowercase	Decode %-encoding and Unicode, then lower-case before checking.
Severity / score	CRITICAL, +5	If it matches, add 5 points to the SQL-injection score.
Tags	attack-sqli	Category, used for scoring groups, dashboards and exclusions.
Action (implicit)	pass + score	In anomaly mode the rule only adds points; a later rule compares the total to the threshold and decides.
Scope (engine extension)	host, path prefix, method	Which sites / URLs / methods this rule applies to – used by staged evaluation.

3. Architecture Diagram

The architecture is shown at two levels: a simple overview (Figure 1) and the full design (Figure 2). Figure 3 shows how staged evaluation shrinks the work, and Figure 4 shows a rule's lifecycle.

3.1 High-level overview

The big picture: every request is inspected against rules, scored, and given one action

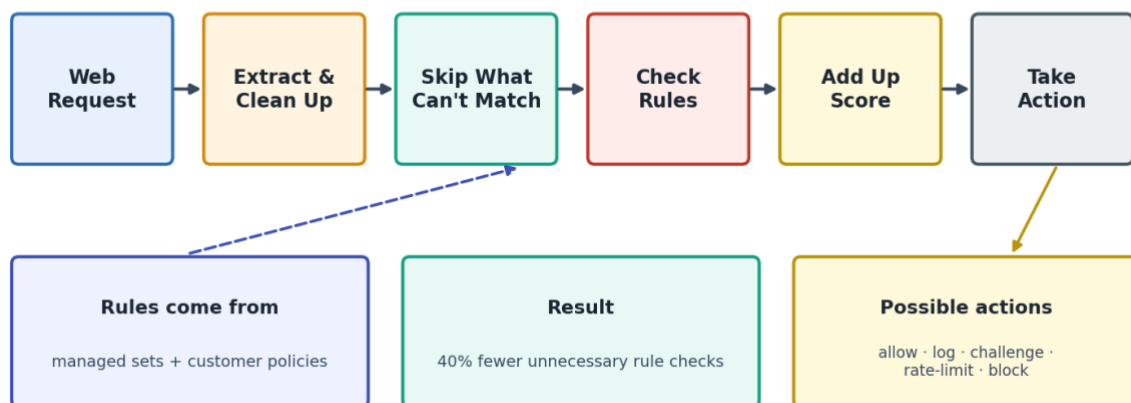


Figure 1 – The big picture: inspect, skip what cannot match, check, score, act

How to read Figure 1: read left to right along the top row. A web request arrives; its parts are extracted and cleaned; rules that cannot possibly match are skipped; the remaining rules are checked; matching rules add to a score; and one action is taken. The blue dashed arrow shows rules flowing in from managed sets and customer policies. The yellow arrow points to the five possible actions. The middle box is the headline result.

3.2 Detailed architecture

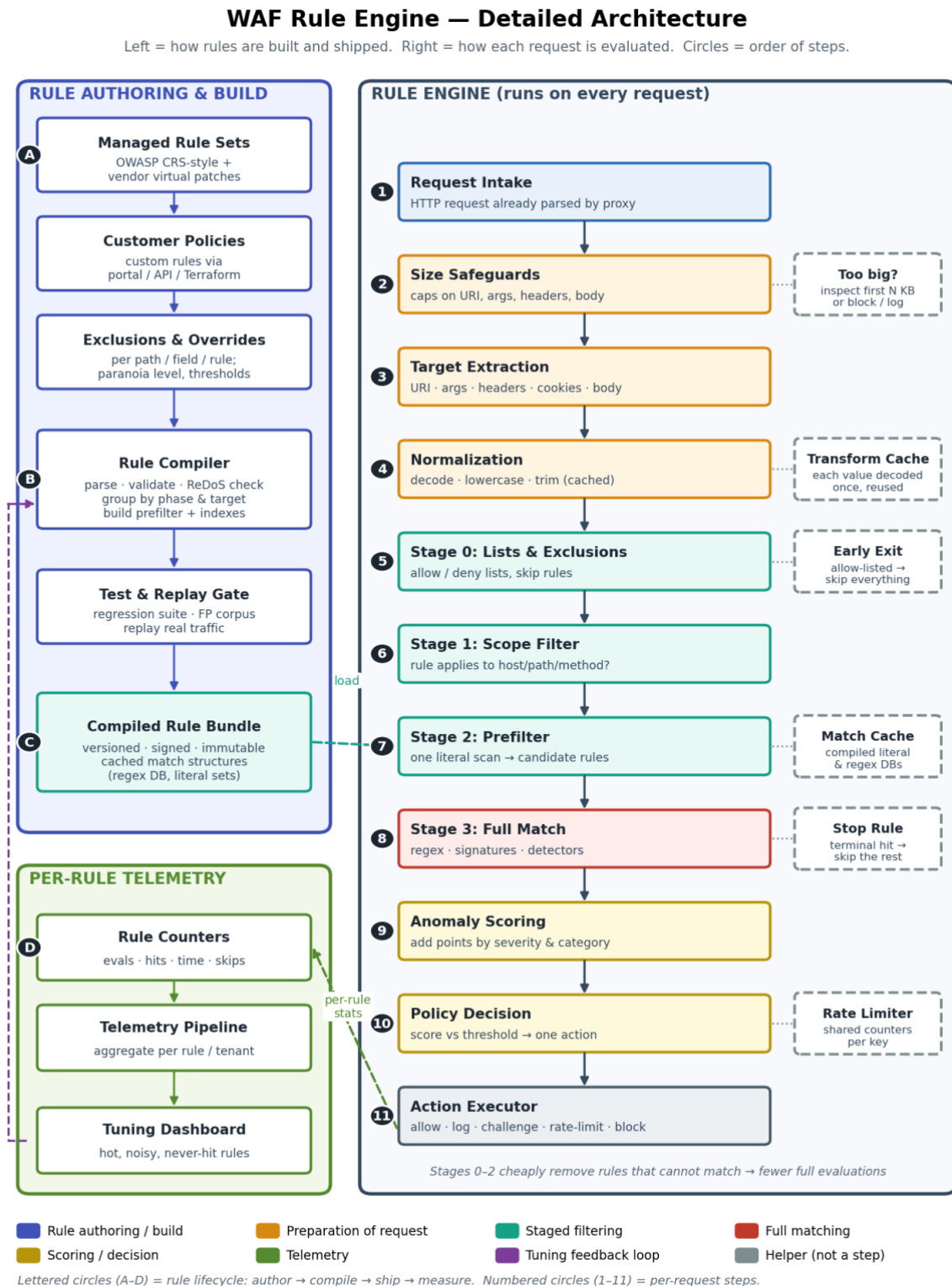


Figure 2 – Detailed architecture of the WAF rule engine

3.3 Reading the detailed diagram, element by element

Visual conventions

Visual element	What it means
Left blue panel “Rule Authoring & Build”	Everything that happens before a request arrives: writing, compiling, testing and packaging rules. Runs only when rules change.
Left green panel “Per-Rule Telemetry”	Measurement of every rule’s behaviour, feeding dashboards and tuning.
Right grey panel “Rule Engine”	The code that runs on every request, steps 1–11.
Lettered circles A–D	The rule lifecycle: A = rules written, B = compiled, C = shipped as a bundle, D = measured.
Numbered circles 1–11	The order in which a single request moves through the engine.
Box colours	Blue = rule authoring; orange = preparing the request; teal = staged filtering (cheap skipping); red = full (expensive) matching; yellow = scoring and decision; green = telemetry; grey = the action. The legend repeats this.
Dashed grey boxes on the right	Helpers attached to a step (size check, transform cache, early exit, match cache, stop rule, rate limiter). They are not extra steps.
Dotted grey line	Connects a step to its helper.
Teal dashed arrow “load”	The compiled rule bundle being loaded into the engine’s match structures.
Green dashed arrow “per-rule stats”	Every request sends counters (evaluated, skipped, matched, time) for each rule to telemetry.
Purple dashed line (far left)	The tuning feedback loop: dashboard insights go back into the compiler (reorder, rescope, fix or retire rules).
Italic line at bottom of engine	The key idea: stages 0–2 cheaply remove rules that cannot match, so fewer full evaluations are needed.

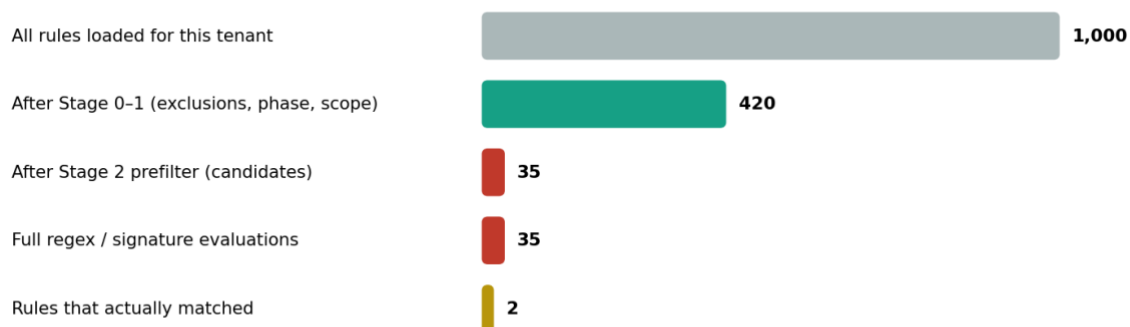
Walking the diagram, element by element

#	Element	What happens there (simple terms)
A	Managed Rule Sets	Rules maintained by the vendor’s security team: an OWASP CRS-style base set plus “virtual patches” for newly discovered vulnerabilities.
A	Customer Policies	Each customer’s own rules, created in a portal, via API or as code (Terraform). E.g. “block POSTs to /admin from outside our office IPs”.
A	Exclusions & Overrides	Exceptions and settings: skip rule X on field Y at path Z; choose paranoia level; set score thresholds and actions.
B	Rule Compiler	Turns all of the above into an optimized form: checks syntax, rejects dangerous regexes, groups rules by phase and target, builds prefilter structures and indexes.

#	Element	What happens there (simple terms)
B	Test & Replay Gate	Runs the compiled rules against attack test suites, a false-positive corpus and replayed real traffic. Bad bundles never ship.
C	Compiled Rule Bundle	A versioned, signed, read-only package holding the ready-to-use match structures. Cached in memory by every engine instance.
1	Request Intake	The engine receives a request already parsed by the proxy (method, URL, headers, body stream).
2	Size Safeguards	Checks lengths and counts against limits. If something is too big, the configured behaviour applies (inspect only the first N KB, log, or block).
3	Target Extraction	Splits the request into named targets: URI path, each query argument, each header, each cookie, and body fields (form, JSON, XML, multipart).
4	Normalization	Applies the transformations rules need (decode, lowercase, etc.). Results are cached so the same value is never transformed twice the same way.
5	Stage 0: Lists & Exclusions	Checks allow/deny lists and removes excluded rules or targets. An allow-listed request can exit early.
6	Stage 1: Scope Filter	Keeps only rules that apply to this phase, host, path, method and content type, using pre-built indexes.
7	Stage 2: Prefilter	One fast scan for the literal words each remaining regex needs. Only rules whose words appear become candidates.
8	Stage 3: Full Match	Runs the real regex, phrase-list, signature and SQLi/XSS detector checks on the candidates only. A terminal rule can stop evaluation.
9	Anomaly Scoring	Adds points from matching rules, by severity and category.
10	Policy Decision	Compares scores to thresholds and applies explicit rule actions to choose one final action. Uses the rate limiter when needed.
11	Action Executor	Carries out allow, log, challenge, rate-limit or block, and records the result.
D	Rule Counters → Pipeline → Dashboard	Per-rule statistics are aggregated and shown as hot, noisy and never-matching rules, driving tuning.

3.4 Staged evaluation funnel

Staged evaluation: how the rules checked for one request shrink at each stage (illustrative)



Cheap checks first; expensive regex only on the few candidates that could possibly match.

Figure 3 – How the number of rules evaluated for one request shrinks at each stage (illustrative)

How to read Figure 3: each bar is the number of rules still “in play” for one request. Out of 1,000 loaded rules, only about 420 are relevant after exclusions and scope checks; the prefilter narrows these to about 35 candidates; only those 35 get an expensive regex or signature check; and just 2 actually match. Before staged evaluation, far more rules reached the expensive step.

3.5 Rule lifecycle

How a rule (managed or customer-defined) goes from idea to enforcement

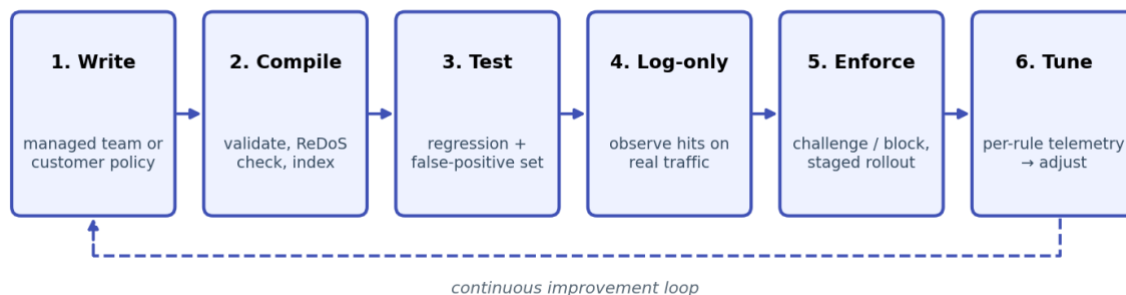


Figure 4 – How a rule goes from idea to enforcement

How to read Figure 4: a rule is written, compiled and validated, tested against attack and false-positive suites, run in log-only mode on real traffic, then enforced through a staged rollout, and finally tuned using per-rule telemetry. The dashed line shows that tuning feeds back into new versions of the rule. Sections 5.1 and 5.2 explain this in full.

4. Components Explained in Detail

Each component follows the same layout: a plain-language summary, what it is, why it is needed, how it works step by step, the small details that matter, what happens when things go wrong, and how it contributed to the result.

4.1 Rule sources and policy model

4.1.1 Managed rule sets

In simple terms

Ready-made protection written and maintained by the vendor's security team, so customers are protected on day one without writing anything.

What it is: Collections of rules in the ModSecurity/OWASP CRS style covering common attack classes (SQL injection, cross-site scripting, remote code execution, local/remote file inclusion, path traversal, protocol violations, scanner detection), plus vendor-specific rules and emergency “virtual patches” for new vulnerabilities (CVEs).

Why it is needed: Most customers do not have security experts to write rules. Managed rules give broad protection and are updated centrally when new threats appear.

How it works, step by step:

1. The security team writes or imports rules and groups them into versioned rule sets (e.g. “Core 4.2”, “WordPress”, “Log4Shell patch”).
2. Each rule carries an ID, category tags, a severity, a paranoia level and a default action.
3. Customers choose which sets to enable and at which paranoia level, and can override actions for individual rules.
4. New versions go through compile → test → log-only → staged rollout (Section 5.1).

Minute details that matter:

- Rules are grouped by paranoia level so customers can tighten protection gradually.
- Every managed rule has positive tests (attacks it must catch) and negative tests (legitimate traffic it must not flag).
- Customers can pin a version or auto-upgrade.

Contribution to the 40% reduction / product value

Rules were enriched with **scope metadata** (which phases, targets, content types and technologies they apply to) and **required literals**, which the staged evaluator uses to skip them safely.

4.1.2 Customer-defined policies and custom rules

In simple terms

Each customer can add their own rules and settings for their own website – like adding house rules on top of the standard ones.

What it is: A per-customer (per-tenant) policy object that includes: which managed sets are enabled, custom rules, exclusions, anomaly thresholds, action overrides, rate-limit rules and allow/deny lists. Authored via a UI, a REST API or infrastructure-as-code (e.g. Terraform).

Why it is needed: Every application is different: an admin area needs stricter rules, an API may need rate limits, and a content editor needs HTML to be allowed in one field.

How it works, step by step:

1. Customer writes a rule in a friendly expression language or in SecRule-compatible syntax (e.g. “if path starts with /admin and IP not in office_ips then block”).
2. The API validates syntax, field names and limits immediately and returns clear errors.
3. The policy is saved as a new version with an audit log entry.
4. The compiler builds a bundle for this tenant, and it is deployed (optionally in log-only mode first).

Minute details that matter:

- Customer rules run in a **sandboxed** way: limits on count, regex complexity, and per-request evaluation time, so one tenant cannot slow down others.
- Custom rules can use the same targets, transformations, operators and actions as managed rules.
- Rule priority is explicit: allow-list and custom rules can be placed before or after the managed set.

Contribution to the 40% reduction / product value

Customer rules also go through the compiler, so they automatically benefit from scope indexing, prefiltering and caching.

4.1.3 Exclusions and overrides

In simple terms

Precise exceptions that turn a specific rule off for a specific place – so legitimate features stop being blocked without turning security off everywhere.

What it is: Structured exceptions such as: “disable rule 942100 for the parameter ‘comment’ on /blog/post”, “don’t inspect cookie ‘session_blob’”, or “use paranoia level 1 on /api/legacy”.

Why it is needed: Generic rules inevitably flag some legitimate traffic (e.g. a code-sharing site where users post SQL). Without exclusions, customers would have to disable whole rules or the whole WAF.

How it works, step by step:

1. Exclusions are defined by conditions (host, path, method) and effects (remove rule / rule tag / target from evaluation).
2. At compile time, exclusions that are always true for a scope are applied to the index directly (the rule simply is not in that scope's list).
3. At runtime, conditional exclusions are applied in Stage 0 before any expensive work.
4. The engine logs when an exclusion prevented a match, so security teams can audit them.

Minute details that matter:

- Exclusions work at three levels: whole rule, rule for one target (e.g. one parameter), or category tag.
- Suggested exclusions can be generated from false-positive telemetry (Section 5.5).

Contribution to the 40% reduction / product value

Applying exclusions **before** matching (instead of matching and then discarding the result, as naive engines do) removed a significant share of wasted evaluations.

4.2 Build-time components

4.2.1 Rule compiler and validator

In simple terms

A translator that turns human-written rules into a super-fast machine-ready form, and a safety inspector that rejects rules that are broken or dangerous.

What it is: An offline build step that parses rules, validates them, and generates optimized, compiled match structures and indexes.

Why it is needed: Interpreting rules from text on every request is slow. Some regexes are dangerous (ReDoS). And the engine needs extra information (scope, required words) to skip rules safely – the compiler works this out once.

How it works, step by step:

1. **Parse** each rule into a structured form: targets, transformations, operator, actions, metadata.
2. **Validate:** syntax, known field names, legal actions, no conflicting IDs, sensible limits.
3. **Regex safety check:** analyse each regex for catastrophic backtracking; test with adversarial inputs; convert to a linear-time engine where possible; reject or flag unsafe patterns.
4. **Extract required literals:** for each regex, compute the words that must appear for it to match (e.g. “union” and “select”). These feed the prefilter.

5. **Group and index:** organize rules by phase → target → transformation chain, and by scope (host, path prefix, method, content type).
6. **Compile match structures:** build a multi-pattern literal automaton (e.g. Aho-Corasick or Hyperscan database) per target group; pre-compile each regex; build hash sets for phrase lists (@pm) and IP trees for @ipMatch.
7. **Package** everything into a versioned bundle.

Minute details that matter:

- Rules whose regex has no extractable literal (e.g. purely character-class patterns) are marked “always candidate” so they are never skipped wrongly.
- Deterministic output: the same input rules always produce the same bundle, which makes testing reliable.
- Compile time is seconds, even for thousands of rules, so urgent patches ship fast.

Contribution to the 40% reduction / product value

The compiler is where most of the 40% reduction is made possible: scope indexes (Stage 1) and required-literal prefilters (Stage 2) are both generated here.

4.2.2 Test and replay gate

In simple terms

A dress rehearsal: every new set of rules is tried against known attacks and real normal traffic before it is allowed to protect anyone.

What it is: Automated testing that includes the rule regression suite (CRS-style test cases), a false-positive corpus of legitimate requests, a performance test, and replay of sampled real traffic.

Why it is needed: A rule that misses attacks is dangerous; a rule that blocks real users is just as damaging to a commercial product; a slow rule hurts every customer.

How it works, step by step:

1. Run all positive test cases: each attack sample must be detected by the intended rule.
2. Run the false-positive corpus: legitimate requests (forms, JSON APIs, rich text, uploads) must not exceed the threshold.
3. Run performance tests: per-rule and total evaluation time must stay within budget.
4. **Equivalence check** for optimizations: compare decisions of the optimized engine with a reference (un-staged) engine on millions of requests – they must match exactly.
5. Replay recent real traffic and compare block rates with the current version; large unexplained changes stop the release.

Minute details that matter:

- The equivalence check is what proves the 40% of skipped evaluations were truly unnecessary.

4.2.3 Compiled rule bundle and match-structure cache

In simple terms

The finished, ready-to-use package of rules. It is built once and then kept in memory, so no request ever waits for rules to be prepared.

What it is: A versioned, signed, immutable artifact containing indexes, compiled regexes, literal automata, phrase sets and metadata. Engines keep compiled bundles in an in-memory cache keyed by (bundle version, tenant).

Why it is needed: Building match structures is expensive. Doing it per request – or even per worker – wastes CPU and memory.

How it works, step by step:

1. The bundle is distributed to all engine instances.
2. Each engine verifies the signature and loads it once; all worker threads share it read-only.
3. Shared managed-rule structures are built once and re-used across all tenants using the same rule-set version; only the small tenant-specific parts are separate.
4. When a new version arrives, it is loaded next to the old one and swapped in atomically; the old one is released once unused.
5. Regex objects, literal automata and phrase sets are looked up directly by pointer during evaluation – no parsing, no compilation at request time.

Minute details that matter:

- Per-worker scratch space (needed by some regex engines) is allocated once and re-used.
- Least-recently-used eviction handles memory for rarely-active tenants; hot tenants stay cached.
- Sharing managed structures across tenants keeps memory use low even with thousands of customers.

Contribution to the 40% reduction / product value

Removed repeated setup work and made the prefilter possible at request time with near-zero overhead.

4.3 Request-time components

4.3.1 Request-size safeguards

In simple terms

Size limits that stop huge or oddly-shaped requests from making the engine do an unreasonable amount of work.

What it is: Configurable limits on: URI length, number and size of arguments, number and size of headers and cookies, body size, depth of JSON/XML nesting, number of multipart parts, and the maximum number of body bytes actually inspected.

Why it is needed: Attackers can send enormous or deeply nested requests to exhaust the WAF, or hide an attack after the inspected portion. Without limits, one request could consume as much work as thousands of normal ones.

How it works, step by step:

1. Before extraction, check lengths and counts against limits (cheap integer comparisons).
2. If a hard limit is exceeded, apply the configured action: block (e.g. HTTP 413), or log and continue.
3. For bodies, inspect up to a configured maximum (e.g. first 128 KB); the policy decides what to do with the rest (pass-through, or block if the site never expects large bodies).
4. Track nesting depth while parsing JSON/XML and stop at the limit.
5. Record which safeguard was triggered in telemetry.

Minute details that matter:

- Limits can differ per route: an upload endpoint allows large bodies but may skip deep text inspection of binary files.
- Content-type awareness: binary uploads (images, zips) are not run through text regexes – a large saving.
- Protocol-violation rules (e.g. too many arguments) are themselves cheap signals that add to the anomaly score.

If it breaks: If a safeguard stops inspection early, the engine never silently allows; it follows the configured fail-open or fail-closed choice and logs it.

Contribution to the 40% reduction / product value

Capping inspected bytes and skipping text rules on binary content removed a large volume of evaluations on big requests and made worst-case cost bounded.

4.3.2 Target extraction and body parsers

In simple terms

Takes the request apart into clearly labelled pieces – address, each parameter, each header, each cookie, each field of the body – so rules can look at exactly what they need.

What it is: Code that builds the collections rules refer to: REQUEST_URI, REQUEST_FILENAME, ARGS_GET, ARGS_POST, ARGS_NAMES, REQUEST_HEADERS, REQUEST_COOKIES, REQUEST_BODY, FILES, XML, JSON fields, and more.

Why it is needed: Rules must be precise (“only the ‘q’ parameter”) and exclusions must be precise (“not the ‘comment’ field”). This requires the request to be split into named pieces.

How it works, step by step:

1. Split the URI into path and query string; parse query arguments into name/value pairs.
2. Collect headers (names lower-cased for lookup) and parse the Cookie header into individual cookies.
3. Choose a body parser from the Content-Type: URL-encoded form, multipart (fields and file metadata), JSON (flatten to paths like user.address.city), or XML.
4. Store each value as a reference to the original bytes plus its name, avoiding copies.
5. Extract **lazily**: a collection is only built if some remaining rule actually needs it.

Minute details that matter:

- Duplicate parameter names (HTTP parameter pollution) are preserved and all copies are inspected.
- Malformed bodies are flagged (a signal in itself) rather than silently ignored.
- Lazy extraction means, for example, cookies are never parsed if no remaining rule targets cookies.

Contribution to the 40% reduction / product value

Lazy, target-aware extraction avoids work for collections no rule needs after staged filtering.

4.3.3 Normalization and transformation engine (with cache)**In simple terms**

Unwraps disguises so attacks look the same however they are written – and remembers the cleaned-up result so it is never done twice.

What it is: An engine that applies ModSecurity-style transformation chains (urlDecodeUni, htmlEntityDecode, lowercase, removeNulls, compressWhitespace, removeComments, normalizePath, base64Decode, cmdLine, etc.) and caches the results per (value, chain).

Why it is needed: Attackers write the same attack in many forms: %53ELECT, SeLeCt, SEL/**/ECT. Many rules share the same transformation chain, so doing it separately for each rule multiplies work.

How it works, step by step:

1. For each target value, each distinct transformation chain needed by remaining rules is identified (the compiler already grouped rules by chain).
2. The chain is applied step by step; each step's output is cached, so chains that share a prefix (e.g. urlDecode → lowercase and urlDecode → lowercase → compressWhitespace) re-use earlier work.
3. If a step does not change the value (e.g. lowercase on an already lowercase value), the original reference is kept – no copy.
4. All rules using that chain on that value then read from the cache.

Minute details that matter:

- Cache lives only for the duration of one request (no cross-request data leakage).
- Decoding is done safely with limits to prevent decode-bomb style inputs.
- Transformations are fast, byte-oriented routines; common ones use SIMD.

Contribution to the 40% reduction / product value

Previously the same value could be re-decoded dozens of times; caching made transformation cost roughly proportional to distinct chains instead of rules.

4.3.4 Staged evaluation scheduler (Stages 0–3)**In simple terms**

The traffic controller of the engine: it runs the cheapest checks first and throws out every rule that cannot possibly match, so only a few rules get the expensive check.

What it is: The component that decides which rules run, in which order, on which targets. It implements four stages: Stage 0 (lists and exclusions), Stage 1 (scope filter), Stage 2 (prefilter), Stage 3 (full match).

Why it is needed: A naive engine runs every rule against every target and filters results afterwards. Most of that work cannot change the outcome. This was the main source of the 40% unnecessary evaluations.

How it works, step by step:

1. **Stage 0 – Lists & exclusions:** check allow list (early exit with allow), deny list (early exit with block), and remove excluded rules/targets for this request.
2. **Stage 1 – Scope filter:** using pre-built indexes, keep only rules for the current phase whose scope matches this host, path, method and content type, and whose targets actually exist in this request (e.g. no body → no body rules).
3. **Stage 2 – Prefilter:** scan each target once with the combined literal automaton; mark rules whose required literals appeared as candidates. Rules with no literals are always candidates.
4. **Stage 3 – Full match:** evaluate candidates in priority order (cheap operators like @streq and @pm before @rx; high-value rules first).
5. **Short-circuit:** if a terminal rule fires (e.g. deny) or the score already exceeds the blocking threshold in “block at threshold” mode, stop evaluating further rules that cannot change the decision.
6. Record, per rule, whether it was skipped (and at which stage), evaluated or matched.

Minute details that matter:

- **Correctness guarantee:** a rule is only skipped when it is provably unable to match or to change the decision. This is enforced by the equivalence tests in the build gate.
- Rules that must always run (e.g. logging or scoring bookkeeping rules) are marked and never skipped.

- Chained rules (rule B only runs if rule A matched) are handled as a unit – if A is skipped, B is skipped too.

Contribution to the 40% reduction / product value

The core of the 40% cut: Stage 0 and 1 remove irrelevant rules; Stage 2 removes rules whose keywords are absent; short-circuiting removes rules after the outcome is decided.

4.3.5 Prefilter (multi-pattern literal matcher)

In simple terms

A quick first glance that finds which rules' "key words" appear in the request, so the engine knows which rules are worth a closer look.

What it is: A multi-pattern string matcher (Aho-Corasick automaton or Hyperscan literal database) built from all required literals of the rules in a target group.

Why it is needed: Regex evaluation is the most expensive operation. Most regexes contain words that must be present for any match – if those words are absent, the regex cannot match and need not run.

How it works, step by step:

1. At compile time, each regex's required literals are extracted (e.g. "union" and "select"; or any of "<script", "javascript:", "onerror=").
2. All literals for a target group are merged into one automaton.
3. At request time, each normalized target value is scanned once through the automaton – cost depends on value length, not on the number of rules.
4. Hits are mapped back to rule IDs using a bitmap.
5. A rule becomes a candidate only if all of its required literal conditions are satisfied.

Minute details that matter:

- Literal matching is done on the transformed (normalized) value so disguises do not defeat it.
- Case-insensitive literals are compiled into the automaton directly.
- Short or very common literals (e.g. "a") are not used because they would not filter anything; such rules become "always candidate".

Contribution to the 40% reduction / product value

The single biggest reduction in regex evaluations.

4.3.6 Regex and signature matcher

In simple terms

The detailed inspection: running the actual attack patterns and fingerprints against the few candidate rules.

What it is: Evaluation of operators: @rx (regex), @pm/@pmFromFile (phrase lists), @contains/@beginsWith/@streq, @detectSQLi and @detectXSS (libinjection-style tokenizers that understand SQL/HTML structure), @ipMatch, @validateUtf8Encoding, numeric comparisons, and vendor signatures for known exploits.

Why it is needed: Candidates from the prefilter might match; this step decides for sure.

How it works, step by step:

1. For each candidate rule and target, fetch the cached transformed value.
2. Run the operator using the pre-compiled structure from the bundle (compiled regex, phrase hash set, IP radix tree).
3. Use a linear-time regex engine or a backtracking engine with a strict step limit, so no input can make a rule run for too long.
4. On a match, record the rule ID, matched target and matched data (truncated and masked for logs).
5. Apply the rule's actions: add to anomaly scores, set variables, or trigger an explicit action.

Minute details that matter:

- Per-rule and per-request time budgets: exceeding them is logged as a telemetry event and handled by the configured fail-open/fail-closed policy.
- Signatures for specific CVEs are usually literal-heavy, so they are extremely cheap after prefiltering.

4.3.7 Anomaly scoring

In simple terms

Instead of blocking on the first suspicious sign, the engine adds up points for everything suspicious and only acts when the total is high enough.

What it is: CRS-style anomaly scoring: each matching rule adds points based on severity (e.g. critical = 5, error = 4, warning = 3, notice = 2) to category scores (SQLi, XSS, RCE, LFI, protocol, etc.) and to a total inbound score.

Why it is needed: Single rules produce false positives. Combining evidence is more accurate: one weak signal is ignored, several together are acted on. It also lets customers tune sensitivity with one number.

How it works, step by step:

1. Initialize scores to zero for the request.

2. Each matching rule adds its points to its category and to the total.
3. After the relevant phase, the scores are compared with thresholds configured in the policy (e.g. inbound threshold 5 = block on one critical match; 10 = needs more evidence).
4. The scoring result, with the list of contributing rules, is passed to the decision step.

Minute details that matter:

- Paranoia level controls which rules can contribute at all.
- Early blocking mode: if the total already exceeds the threshold, remaining rules in that phase that cannot change the decision are skipped (a source of savings on attack traffic).
- Scores and contributing rules are always logged, which makes decisions explainable to customers.

4.3.8 Policy decision and action executor

In simple terms

The final judge: it turns scores and rule results into exactly one action and carries it out.

What it is: The component that merges explicit rule actions (e.g. a custom rule that says “block”) with the anomaly-score outcome and the policy’s settings, and executes the chosen action.

Why it is needed: Customers need different outcomes for different situations – block for obvious attacks, challenge for suspected bots, rate-limit for abusive clients, log while testing.

How it works, step by step:

1. Collect: explicit actions from matched rules, anomaly score results, rate-limit state, and policy mode (enforce vs log-only).
2. Apply precedence: allow-list > explicit block > rate-limit > challenge > score threshold > log > allow (order configurable per policy).
3. Execute: **Allow** – pass the request on. **Log** – pass it on but record an event. **Challenge** – return a JavaScript/CAPTCHA page; a passing client receives a signed cookie. **Rate-limit** – increment the counter for the rule’s key (IP, API key, path); if over the limit, return HTTP 429. **Block** – return HTTP 403 with a reference ID.
4. Write a security event with rule IDs, scores, targets and action.

Minute details that matter:

- Log-only (“simulate”) mode lets customers see what a policy would do before enforcing it.
- Rate-limit counters are kept locally in memory and synced in the background, so the check does not wait on the network.
- Responses are pre-built templates, so blocking is cheap.

4.4 Measurement and improvement components

4.4.1 Per-rule telemetry

In simple terms

A counter and stopwatch for every rule, so we know exactly which rules do useful work, which waste time, and which cause false alarms.

What it is: Lightweight per-rule, per-tenant statistics: evaluations, skips (by stage), matches, contribution to blocks, CPU time (sampled), time-budget overruns, and false-positive reports linked to rules.

Why it is needed: Without per-rule data, you cannot see that one rule is responsible for 15% of all CPU, or that another has never matched in a year, or that a third causes most false-positive complaints.

How it works, step by step:

1. The engine increments in-memory counters per rule (per worker, no locks) as it evaluates.
2. Timing is sampled (e.g. 1 in 100 evaluations) to keep overhead tiny.
3. Counters are flushed every few seconds to the telemetry pipeline and aggregated per rule, per rule-set version and per tenant.
4. Dashboards show: top rules by CPU, rules with low match rates relative to evaluations (wasted work), noisy rules (high match, high FP), and never-matching rules.
5. Alerts fire on anomalies, e.g. a rule suddenly matching on 5% of a tenant's traffic (likely a false-positive storm).

Minute details that matter:

- Counters are keyed by rule ID and bundle version so improvements can be compared across releases.
- Customer-facing views show each customer which rules fire on their traffic, helping them write exclusions.

Contribution to the 40% reduction / product value

Telemetry found where unnecessary evaluations came from and proved each fix worked. It turned optimization from guesswork into measurement.

4.4.2 Tuning loop and dashboards

In simple terms

Using the rule statistics to keep improving: reorder, re-scope, fix or retire rules, and suggest exclusions to customers.

What it is: A regular process plus tooling that turns telemetry insights into rule changes.

Why it is needed: Rule sets drift over time: new rules are added, traffic changes, some rules become irrelevant. Continuous tuning keeps the engine lean and accurate.

How it works, step by step:

1. Review top-cost rules: can the regex be simplified, can a required literal be added, can the scope be narrowed (e.g. only for PHP sites)?
2. Review never-matching rules: are they obsolete or just rare but important? Mark obsolete ones for retirement.
3. Review noisy rules: fix the pattern or add default exclusions for known-safe fields.
4. Reorder evaluation priority so rules that most often decide outcomes run first.
5. Ship changes through the normal compile → test → rollout path, then compare telemetry before/after.

5. Workflows Explained End to End

5.1 Managed rule release lifecycle

1. **Write:** a security researcher writes a new managed rule (e.g. a virtual patch for a new CVE) with ID, targets, transformations, operator, severity, tags, paranoia level and scope metadata.
2. **Review:** another researcher reviews it; test cases (attack samples and legitimate look-alikes) are added.
3. **Compile:** the compiler validates syntax, checks the regex for ReDoS risk, extracts required literals, and places the rule in the right phase/target/scope indexes.
4. **Test:** regression suite, false-positive corpus, performance budget and equivalence checks all pass.
5. **Log-only rollout:** the new bundle version is deployed with the rule in log-only mode; telemetry shows how often it matches on real traffic and on which tenants.
6. **Enforce:** if hits look like real attacks and FPs are negligible, the rule is switched to scoring/blocking, rolled out in waves (canary → percentage → all).
7. **Tune:** telemetry is monitored; if an FP pattern appears, the rule is refined or a default exclusion is added.

5.2 A customer creates a custom rule

Example

A customer wants: “rate-limit POST /api/login to 10 requests per minute per IP, and block any request to /admin that does not come from our office IP range.”

1. The customer adds two rules in the portal (or via API/Terraform).
2. The API validates field names, IP range format, limit values and checks the customer’s rule quota.
3. The policy is saved as version N+1 with an audit entry.
4. The compiler builds a new tenant bundle: the /admin rule is indexed under scope “path prefix /admin” so it is only considered for those URLs; the login rule is indexed under “POST /api/login”.
5. The customer chooses log-only first; the bundle is deployed and swapped in atomically on every engine.
6. The customer views the log to confirm the rules behave as expected, then switches them to enforce.
7. Because of scope indexing, these rules cost nothing for the other 99% of the customer’s URLs.

5.3 Evaluating a normal request (end to end)

Example

GET /products?category=shoes&sort=price with normal browser headers and cookies, no body.

Step	What happens technically	In simple terms
1	Engine receives the parsed request and the tenant's cached bundle pointer.	The request arrives with the right rule book already open.
2	Size safeguards: URI 38 bytes, 2 args, 14 headers, no body – all within limits.	The bag is a normal size.
3	Targets extracted lazily: path, 2 args, headers, cookies. No body collections are built.	Pockets are listed; there is no suitcase.
4	Needed transformation chains applied once per value and cached.	Items are unwrapped once.
5	Stage 0: not on allow or deny list; no exclusions apply to /products.	No special pass, no ban.
6	Stage 1: phase-2 body rules, file-upload rules, /admin-only and POST-only rules dropped. ~1,000 → ~420 rules.	Checks that don't apply to this traveller are skipped.
7	Stage 2: one literal scan per value; "shoes", "price" and header values contain no attack keywords. ~420 → ~35 candidates (mostly literal-less rules).	A quick X-ray shows nothing suspicious.
8	Stage 3: ~35 cheap-to-moderate checks run; none match.	A few routine hand checks, all clear.
9	Anomaly score = 0.	No suspicion points.
10	Decision: ALLOW.	Cleared.
11	Request passed on; per-rule counters updated (evaluated / skipped by stage).	Waved through; the records are updated.

5.4 An attack is detected, scored and blocked

Example

POST /search with body: q=1%27+UNION%2F%2A%2A%2FSELECT+password+FROM+users--

1. Size safeguards pass (small body).
2. Target extraction parses the URL-encoded body into ARGS_POST:q.
3. Normalization (urlDecodeUni → lowercase → removeComments) turns the value into "1' union select password from users--".
4. Stage 0: no allow list or exclusion covers /search → q.
5. Stage 1: SQLi rules for ARGS in phase 2 remain in scope.
6. Stage 2: prefilter finds "union", "select", "from" and a quote character; several SQL-injection rules become candidates.
7. Stage 3: the UNION-SELECT regex matches (+5, critical); the SQLi tokenizer detector confirms a SQL structure (+5); a comment-sequence rule matches (+3).
8. Anomaly scoring: SQLi score 13, total inbound score 13, threshold 5 – exceeded. With early blocking enabled, remaining phase-2 rules that cannot change the outcome are skipped.

9. Decision: BLOCK. The action executor returns HTTP 403 with a reference ID.
10. A security event lists the rule IDs, the matched target (ARGS_POST:q), a masked excerpt, the scores and the action. Per-rule counters record the matches.

5.5 A false positive is found and fixed with an exclusion

1. A customer's developer forum lets users post code. Posts containing SQL examples are blocked by rule 942100 on field ARGS_POST:body at /forum/post.
2. Per-rule telemetry shows 942100 matching on 3% of this tenant's /forum/post requests with no other attack signals – a classic FP pattern. The customer also reports it via the dashboard.
3. The platform suggests an exclusion: "remove rule 942100 from target ARGS_POST:body for path /forum/post".
4. The customer accepts; the policy is updated; the compiler applies it to the scope index.
5. The rule still protects every other field and path. At /forum/post, the rule is removed in Stage 0/1, so it is not evaluated at all (saving work as well).
6. Telemetry confirms the FP rate drops to zero and no attack detections were lost elsewhere.

5.6 An oversized or malformed request

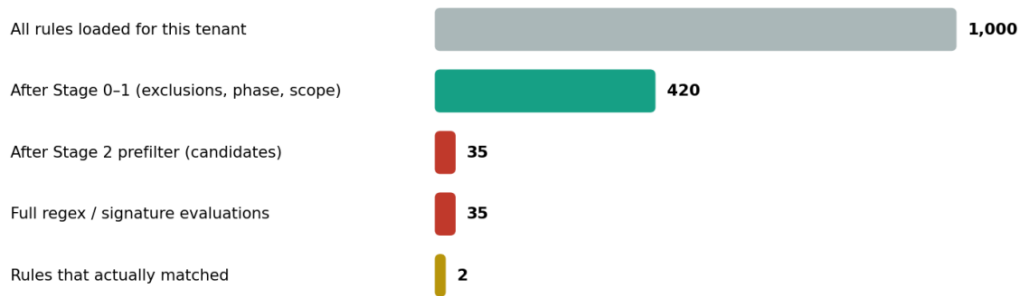
1. An attacker sends a 50 MB JSON body with 10,000 levels of nesting, hoping to exhaust the WAF or hide an attack deep inside.
2. Size safeguards see the body exceeds the route's maximum inspected size (e.g. 128 KB) and max body size (e.g. 10 MB).
3. Per policy, the request is blocked with HTTP 413 (or, on a route allowing large bodies, only the first 128 KB are inspected).
4. While parsing, the JSON nesting limit (e.g. 64 levels) is hit; the parser stops and raises a "request body malformed / limit exceeded" signal, which adds to the anomaly score.
5. The engine spends a small, bounded amount of work on this request instead of an unbounded amount.
6. Telemetry records which safeguard fired, helping customers tune their limits.

5.7 Challenge and rate-limit actions

1. **Rate-limit:** a custom rule limits POST /api/login to 10 per minute per IP. Each matching request increments the IP's counter. On the 11th, the decision step returns RATE-LIMIT and the executor sends HTTP 429 with a Retry-After header.
2. Counters live in memory and sync in the background across servers, so the check never waits on the network.
3. **Challenge:** a managed rule scores a request as a probable automated scanner (unusual user agent + scanner paths) with a medium score. The policy says "medium score → challenge".
4. The executor returns a lightweight JavaScript challenge page. A real browser solves it and receives a signed cookie; the retried request carries the cookie and is allowed.
5. A simple script cannot solve the challenge, so it never gets through.

5.8 How the 40% reduction was achieved (tuning workflow)

Staged evaluation: how the rules checked for one request shrink at each stage (illustrative)



Cheap checks first; expensive regex only on the few candidates that could possibly match.

Figure 3 (repeated) – Staged evaluation funnel

1. **Measure the baseline:** turn on per-rule telemetry (evaluations, matches, time) across representative tenants and traffic.
2. **Classify evaluations:** for every evaluation, determine whether it could have changed the outcome. Evaluations that could not (wrong phase or target absent, excluded, required keywords absent, decision already final) are “unnecessary”.
3. **Attack the biggest categories first** with the changes listed in the table below.
4. **Prove equivalence:** run the optimized engine and a reference engine side by side on replayed traffic and attack suites; decisions must be identical.
5. **Roll out gradually** and compare per-rule telemetry before and after on live traffic.
6. **Keep it in place:** add CI checks so new rules must declare scope and have extractable literals (or be explicitly marked), preventing the waste from coming back.

Where the reduction came from (illustrative breakdown)

Source of unnecessary work	Technique that removed it	Share of reduction (illustrative)
Rules evaluated on targets that did not exist or were irrelevant (e.g. body rules with no body, upload rules with no files)	Staged evaluation – Stage 1 scope and target-presence filter; lazy extraction	~30%
Regex rules run although their required keywords were absent	Compiled literal prefilter (Stage 2) with cached match structures	~35%
Rules evaluated then discarded by exclusions or path-specific policies	Exclusions applied before matching (Stage 0) and folded into scope indexes at compile time	~15%
Text rules run over huge bodies and binary uploads	Request-size safeguards: inspection caps, content-type awareness, nesting limits	~10%
Rules evaluated after the decision was already final	Short-circuit / early blocking in scoring mode; priority ordering	~5%

Source of unnecessary work	Technique that removed it	Share of reduction (illustrative)
Obsolete or duplicate rules	Per-rule telemetry review; retired or merged rules	~5%

Together these removed about **40% of all rule evaluations** that could not affect the outcome – with detection results identical to the reference engine. Secondary benefits included lower CPU per request, lower and more stable inspection latency, and room to add more rules.

5.9 Safely changing the engine itself

1. Unit tests for every operator, transformation and body parser.
2. Full CRS-style regression suite and vendor rule tests.
3. Differential testing: new engine vs reference engine on millions of recorded requests – any difference in decision blocks release.
4. Fuzzing of parsers and transformations with malformed inputs.
5. Performance benchmarks with budgets per stage.
6. Canary deployment, then staged rollout, with per-rule telemetry compared to the previous version and instant rollback available.

6. Key Design Decisions and Trade-offs

Decision	Choice made	Why	Trade-off accepted
Rule language	ModSecurity/CRS-compatible model plus friendly expressions	Familiar to security teams; can import community rules	Must support many legacy features and edge cases
Evaluation strategy	Staged: cheap filters before expensive matching	Removes work that cannot change the outcome	More complex engine; needs equivalence testing
Prefilter	Required-literal extraction + multi-pattern automaton	Skips most regexes on normal traffic	Rules without literals always run; extraction must be correct
Match structures	Compiled once, cached, shared across tenants	No per-request setup; low memory	Bundle build step on every policy change
Scoring model	Anomaly scoring with thresholds	Fewer false positives; one tuning knob	Harder to explain than simple “rule X blocked”, so detailed logs are needed
Exclusions	Applied before matching, compiled into indexes	Accurate and saves work	Must be audited so they do not become security holes
Size safeguards	Inspect up to a cap; per-route limits	Bounded worst-case cost; protects the engine	Attacks beyond the cap are handled by policy, not inspection
Telemetry	Per-rule, sampled timing, lock-free counters	Enables data-driven tuning	Small overhead and a telemetry pipeline to operate
Multi-tenancy	Sandboxed customer rules with quotas and time budgets	One customer cannot slow others	Limits on how complex custom rules can be

7. Recap: Explaining the Project in Two Minutes

A short spoken version

“I shipped the rule engine at the heart of a commercial WAF. It runs both our managed rule sets – ModSecurity and OWASP CRS-style – and each customer’s own policies. For every request it extracts the URI, query string, headers, cookies and body, normalizes them to defeat encoding tricks, runs regex and signature matching, accumulates anomaly scores, applies exclusions, and ends with one action: allow, log, challenge, rate-limit or block.

The main improvement was efficiency without losing detection. Using per-rule telemetry, I found that a large share of evaluations could never affect the result. I introduced staged evaluation – exclusions and scope first, then a compiled literal prefilter, and only then full regex on the few candidates. I compiled and cached all match structures in versioned bundles shared across tenants, and added request-size safeguards so huge or binary bodies cannot blow up the work. An equivalence test against a reference engine proved decisions were unchanged, and unnecessary rule evaluations dropped by 40%.”

Key takeaways

- **Inspect everything, but cleverly** – every part of the request is covered, yet only relevant rules run.
- **Cheap first, expensive last** – exclusions and scope, then keyword prefilter, then regex.
- **Build once, reuse always** – compiled, cached match structures and transformation results.
- **Bound the worst case** – size and depth limits protect the engine from abuse.
- **Measure every rule** – per-rule telemetry drives tuning and proves improvements.
- **Never trade away detection** – equivalence testing guarantees the skipped work was truly unnecessary.